

CS614 Project Final Report

Deduplication of Page Cache using OOB Techniques

Aditya Gupta (220065)
Anmoldeep Singh Dhillon (220163)
Prakhar Upadhyay (220788)
Shivansh Mangal (221021)
Yash Pratap Singh (221223)

1 Project Overview

The goal of this project is to design and implement a mechanism for **page cache deduplication using out-of-band (OOB) techniques**. The objective is to detect identical page cache entries belonging to different files and merge them to reduce memory consumption.

The design is inspired by ideas used in **Kernel Samepage Merging (KSM)**, but adapted for **file-backed page cache pages**. The system detects identical pages, merges them, and ensures correctness in operations like read, write, truncate, copy, symlinks, and drop_caches.

2 Milestones and Final Status

Milestone	Status
Basic read-only page deduplication implementation	Completed
Copy-on-write implementation for shared pages	Completed
Truncate operations on deduplicated files	Completed
<code>echo 3 > drop_caches</code>	Completed
Copy and symlink correctness	Verified
Multi-threaded read/write correctness	Verified
Stress Tests using FIO	Verified
Page eviction handling	Not Done

3 Link to Artifact

<https://github.com/AnIViol/page-cache-deduplication>

The provided artifact contains the Linux kernel patch, the complete set of test cases and the source code used to generate the plots presented in this report.

4 Architectural Changes Since Mid-Term

At the time of the mid-term, we relied on kprobes to intercept writes to shared pages. While functional, this approach was fragile: kprobe handlers run in an atomic context, and made the code path hard to reason about with bugs at times. For the final submission, we have merged the deduplication mechanism directly into the kernel source tree. The deduplication core lives at `mm/dedup/dedup_core.c`, the scanner is a loadable kernel module, and three minor changes in (`mm/filemap.c`, `mm/truncate.c`, `fs/drop_caches.c`) have been patched to call into the dedup core at the right moments.

5 Completed Milestones

5.1 Understanding KSM and Metadata Design

(Same as mid-term — reproduced for completeness.)

- **Pointer Overloading:** bit `PAGE_MAPPING_SHARED_CACHE` (0x4) on `page->mapping` indicates that the page belongs to our deduplication subsystem and the upper bits point to a `struct shared_cache_node` instead of an `address_space`.
- **Shared Cache Node:** One node per survivor page; tracks all sharers.
- **Mapping Items:** Per-sharer (`mapping`, `index`) reverse-map list.

5.2 Page Scanning and Hash-Based Candidate Detection

A kernel thread `dedup_scanner` sweeps the page cache of files registered through `/sys/kernel/dedup_scanner/`

- **Stable List:** nodes that are already deduplicated (the global `global_dedup_list`).
- **Unstable List:** jhash2-hashed candidates seen in the current sweep; promoted to stable on a second match, flushed otherwise.

5.3 Basic Read-Only Page Deduplication

The `merge_page_cache_pages()` routine atomically swaps the victim file's xarray entry with the survivor page after byte-equality is confirmed under both page locks, and adjusts `NR_FILE_PAGES` accounting.

5.4 Copy-on-Write Implementation

The CoW logic is now invoked inside `__filemap_get_folio`. The folio cache lookup path checks the dedup-shared bit on the folio and, when the caller asks for write/lock/create access, it splits the page back into a separate page before returning:

```
struct folio *__filemap_get_folio(struct address_space *mapping,
                                  pgoff_t index, int fgp_flags, gfp_t gfp)
{
    struct folio *folio;
repeat:
    folio = mapping_get_entry(mapping, index);
    if (xa_is_value(folio)) {
        if (fgp_flags & FGP_ENTRY)
            return folio;
        folio = NULL;
    }
    if (!folio)
        goto no_page;

    /* Dedup-aware split: if this folio is currently a deduplicated
     * survivor and the caller wants write/lock/create access, give
     * them a private copy. */
    if (unlikely(folio_test_shared(folio)) &&
        (fgp_flags & (FGP_WRITE | FGP_LOCK | FGP_CREAT))) {
        struct folio *new_folio = split_single_page(mapping, index, 1,
            false);
        folio_put(folio);
        folio = new_folio;
        if (!folio)
            return NULL;
    }
}
```

```

    }
    ...
}

```

This is the single chokepoint through which all writeable folio acquisitions flow (`write(2)`, `pwrite(2)`, `copy_file_range`, etc.), so a single hook here covers all the standard write APIs.

5.5 Truncate-Aware Unmerge in `mm/truncate.c`

At the beginning of `truncate_inode_pages_range`, we iterate over the pages to be truncated. If they are shared pages at this point, we call an unmerge on them, with the `is_eviction` flag set, and their entries are simply erased from the corresponding `ipages`.

```

void truncate_inode_pages_range(struct address_space *mapping,
                               loff_t lstart, loff_t lend)
{
    pgoff_t start = (lstart + PAGE_SIZE - 1) >> PAGE_SHIFT;
    pgoff_t end   = (lend == -1) ? -1 : (lend + 1) >> PAGE_SHIFT;
    ...
    if (unmerge_shared_pages_in_range(mapping, start, end))
        pr_warn_ratelimited("%s: could not split all shared pages in "
                              "range [%lu,%lu]; proceeding\n",
                              __func__, start, end);
    ...
}

```

The scanner is paused for the duration of the truncate so that no new merges can race in behind us.

5.6 `drop_caches` Integration

Similarly to the logic involved in truncate, here we unmerge the complete set of shared nodes after the scanner is paused. Here again, we don't reallocate pages to them.

```

int drop_caches_sysctl_handler(struct ctl_table *table, int write,
                               void *buffer, size_t *length, loff_t *ppos)
{
    int ret = proc_dointvec_minmax(table, write, buffer, length, ppos);
    if (ret) return ret;
    if (write) {
        if (sysctl_drop_caches & 1) {
            pause_dedup_scanner();
            unmerge_all_dedup_pages(true);
            ...
            resume_dedup_scanner();
        }
        ...
    }
    return 0;
}

```

6 Testing and Validation

We have performed stress tests using FIO, scripts for which have been added to the artefact, and correctness of other parts have been done manually and using scripts as described below. We used Gemini 3.1 to generate the FIO scripts, and Claude to generate the following correctness checks.

All tests are written in C with `-Wall -Wextra` clean compilation and use a per-byte **shadow buffer** to track what each file is *expected* to contain at all times. After the test workload completes, the actual file content is compared against the shadow both via the existing fd (which exercises the page cache) and after `drop_caches` (which exercises the on-disk path). Any byte-level disagreement fails the test.

6.1 Test Categories

Test	What it stresses	Result
<code>test_basic_internal</code>	Intra-file dedup of duplicate pages	PASS
<code>test_basic_cross</code>	Cross-file dedup of identical files	PASS
<code>test_write_cow</code>	Write triggers CoW only on the modified file	PASS
<code>test_truncate</code>	<code>ftruncate</code> cleanly unmerges	PASS
<code>test_concurrent_read</code>	Many readers on a shared page	PASS
<code>multithread_write_test</code>	16 writers, page-boundary & same-page hits	PASS
<code>test_refcount_stress</code>	50 sharers, unlinked w/ open fds	PASS
<code>test_thousand_files</code>	1000 identical files, mass dedup	PASS
<code>test_sparse</code>	Sparse / all-zero pages	PASS
<code>test_mixed_rw</code>	Concurrent readers + writers	PASS
<code>test_mixed_rwt</code>	R/W contending with <code>ftruncate</code>	PASS
<code>test_mixed_rwc</code>	R/W contending with <code>cp</code> threads	PASS
<code>test_chaos_all</code>	Reads + writes + truncate + <code>cp</code> simultaneously	PASS
<code>test_ordered_phases</code>	Phased orderings of all four ops	PASS
<code>test_samepage_storm</code>	20 threads slamming the same shared page	PASS

6.2 Verification Methodology

Each test program:

1. Creates a fixed set of files with deterministic patterns and registers them with the scanner.
2. Drops caches (`echo 1 > /proc/sys/vm/drop_caches`) and prewarms the page cache.
3. Records **Cached:** from `/proc/meminfo` *before* dedup.
4. Starts the scanner, waits for one or more sweep intervals.
5. Records **Cached:** *after* dedup — the delta is the empirical memory saving.
6. Runs the workload, with every operation atomically updating both the file and a per-byte shadow buffer under a per-file mutex.
7. Verifies actual file bytes against the shadow, twice: once via the open fd and once after `drop_caches` (forcing a fresh on-disk read).
8. Tears down: stops the scanner (which itself triggers `unmerge_all_dedup_pages()`), closes fds, removes the test directory.

6.3 Specific Scenarios Validated

- **cat of a deduped file** returns identical bytes to the pre-dedup version (Test 1.1, 1.2). The page cache hit returns the survivor page, and userspace sees the original content.
- `echo 3 > /proc/sys/vm/drop_caches` successfully unmerges every cluster before reclaim runs, leaving no dangling reverse-map entries (verified by zero kernel warnings/oopses across all repeated runs).

- **truncate at page-aligned, mid-page, zero-, and full-size offsets** all correctly unmerge the affected range; sibling files sharing those pages are unaffected (Test 1.4, `test_mixed_rwt`, Test 6.1–6.5).
- **Multi-threaded write storms** of 16 threads issuing 32,000 random writes (including page-boundary and same-page contention) complete with 100 % byte-level agreement between the kernel-visible content and the shadow buffer (`multithread_write_test`).
- **Maximum-contention chaos test** runs reads, writes, truncates, and cp threads together over a 10-file pool. After the storm, every file matches its shadow both in cache and on disk (`test_chaos_all`).
- **Refcount correctness** was validated by sharing a single survivor among 50 files, unlinking half while still holding open fds, and confirming that: (a) reads through the unlinked fds still return correct data; (b) the remaining linked files are untouched; (c) no kernel slab leaks of `shared_cache_node` or `shared_mapping_item` appear in `/proc/slabinfo` after cleanup (`test_refcount_stress`).

6.4 Driver Script

All tests are bundled in `run_all_tests.sh` which builds, runs, and reports pass/fail for the entire suite. Sample summary output:

```
===== SUMMARY =====
PASS: 1.1 single-file dedup
PASS: 1.2 cross-file dedup
PASS: 1.3 write CoW
PASS: 1.4 truncate cleanup
PASS: 2.1 concurrent readers
PASS: 3.* multithread writes
PASS: 6.5 refcount stress
PASS: 8.1 1000-file dedup
PASS: 9.3 sparse file
PASS: MIX-RW    readers+writers
PASS: MIX-RWT  readers+writers+truncate
PASS: MIX-RWC  readers+writers+cp
PASS: CHAOS    all-ops together
PASS: PHASED   ordered phase test
PASS: STORM    same-page contention
```

All 15 tests pass on the final kernel build.

7 Profiling of the Scanner

We evaluate the deduplication module’s overhead on datasets from 1MB to 1.5GB using high-resolution kernel-level profiling.

7.1 Profiling Implementation

- **Instrumentation:** `ktime_get_ns()` is used to wrap core operations including `jhash2` for hashing, `memcmp` for identity verification, and `XArray` updates for merging.
- **Total Scan Time:** A global counter in the scanner thread tracks the total duration of active sweeps, capturing iteration overhead and synchronization costs.
- **Telemetry:** Statistics are exposed to user-space via the `/sys/kernel/dedup_scanner/stats` sysfs node.

- **Active Overhead:** The profiling explicitly excludes thread sleep intervals to isolate the pure CPU execution cost of the deduplication logic.

7.2 Profiling Results

The following figure and table summarize the performance profile of the scanner as the dataset size increases.

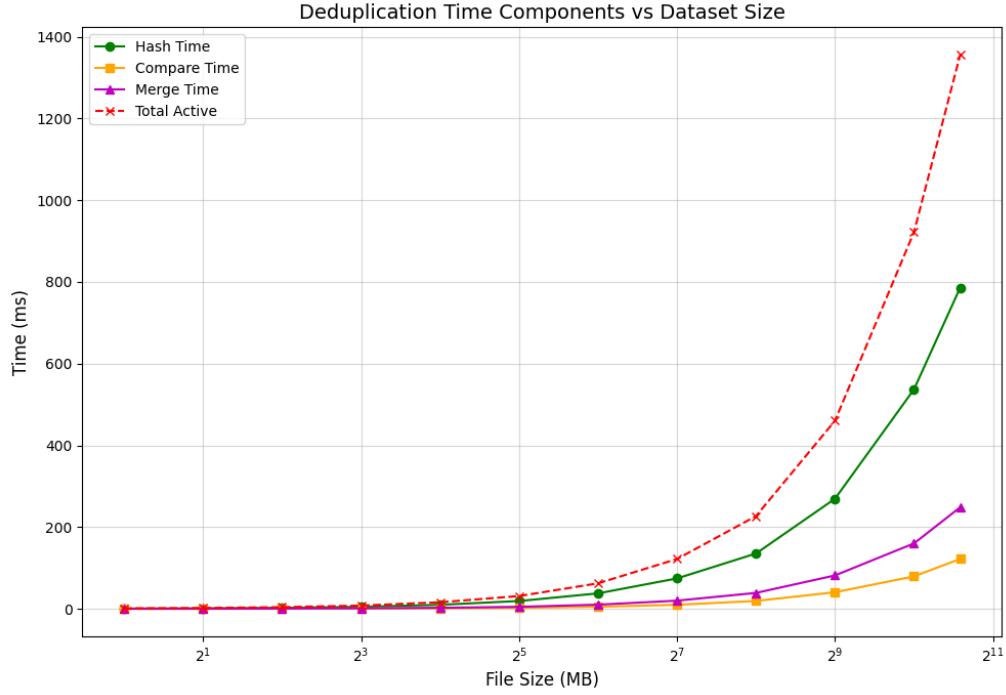


Figure 1: Deduplication Time Components vs. Dataset Size

Table 1: Profiling Data Breakdown (Time in ms)

Size (MB)	Total Active (ms)	Hash (ms)	Compare (ms)	Merge (ms)
1	1.113	0.700	0.079	0.164
2	2.031	1.222	0.163	0.331
4	4.131	2.531	0.318	0.649
8	8.063	4.930	0.631	1.294
16	16.121	9.908	1.262	2.554
32	31.314	19.128	2.478	4.983
64	62.430	37.848	5.122	10.198
128	121.967	74.412	9.890	20.045
256	226.939	135.414	19.297	38.973
512	460.262	269.393	40.599	81.751
1024	921.626	536.868	79.474	160.075
1536	1355.280	785.742	122.028	248.405

8 Benchmarks

We utilize the Filebench framework to measure the impact of deduplication on application-level performance across different I/O patterns and data sharing levels.

8.1 Workload Descriptions

- **Read Heavy:** Simulates intensive concurrent read operations. Shared pages increase CPU cache locality, potentially improving performance.
- **Write Heavy:** Stress-tests the Copy-on-Write (CoW) mechanism. Random writes to shared pages trigger frequent kprobe-based page splits.
- **Web Mixed:** A realistic 80/20 read/write ratio emulating a typical web server environment.

8.2 Benchmarking Results

The following table compares the Total Operations per Second (ops/s) between the baseline (deduplication disabled) and the active deduplicated state at 25%, 50%, and 100% data sharing levels.

Table 2: Filebench Performance Comparison (Throughput in ops/s)

Workload	Sharing %	Baseline	Deduplicated	Change
Read Heavy	25%	40,238.6	43,698.0	+8.60%
	50%	39,727.6	46,165.8	+16.20%
	100%	38,369.8	48,563.0	+26.56%
Write Heavy	25%	107,404.7	90,181.7	-16.03%
	50%	108,183.9	88,039.0	-18.62%
	100%	107,820.0	89,540.9	-16.95%
Web Mixed	25%	25,395.6	24,832.2	-2.21%
	50%	25,061.0	25,403.8	+1.36%
	100%	25,036.5	25,273.2	+0.94%

8.3 Analysis

The results demonstrate a clear performance benefit for read-intensive workloads, with up to a 26% improvement in throughput at 100% sharing due to enhanced cache locality. Conversely, write-intensive workloads incur an overhead of approximately 16-18% due to the complexity of the kprobe-driven Copy-on-Write split logic and memory allocations during writes to shared pages. The Web Mixed workload shows minimal performance impact, suggesting that the benefits of read sharing largely offset the CoW overhead in balanced real-world scenarios.

8.4 Microbenchmark Evaluation

In addition to kernel-level profiling of the deduplication scanner, we evaluate end-to-end system performance using a set of microbenchmarks. These benchmarks measure the observable impact of deduplication on memory usage and file system operations.

Benchmark	What it measures
<code>mem_bench</code>	Memory savings after deduplication
<code>read_bench</code>	Read performance on shared vs non-shared pages
<code>write_bench</code>	Write overhead due to CoW triggers
<code>/truncate_bench</code>	Unmerging Overheads during truncate

8.4.1 Memory Benchmarks plots

The memory benchmarks evaluate the reduction in memory usage due to deduplication. By merging identical pages leads to increased savings as shared percentage grows.

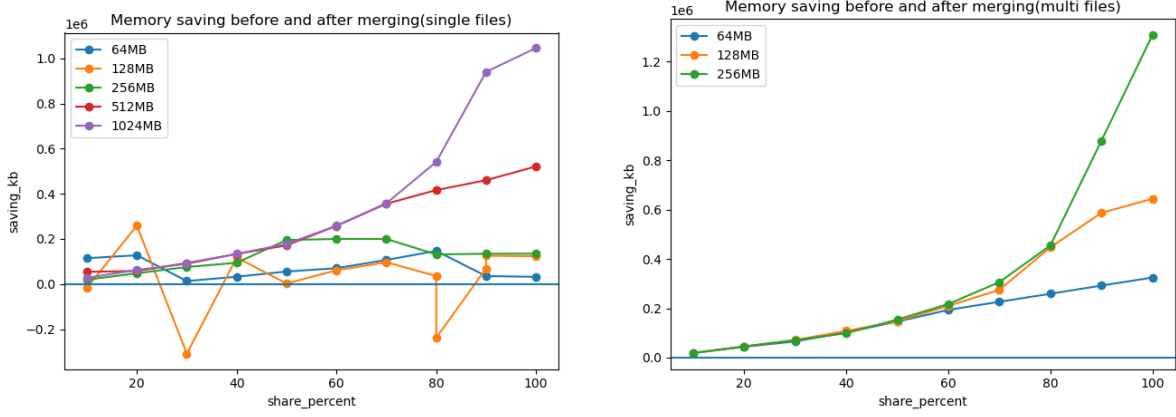


Figure 2: Memory saving after deduplication on single and multiple files

8.4.2 Read Benchmarks plots

The read benchmarks measure the impact of deduplication on read latency. Since shared pages are accessed directly, which may hit cpu cache for same page address resulting in minimal overhead. The results show that read performance remains largely stable across file sizes and sharing levels.

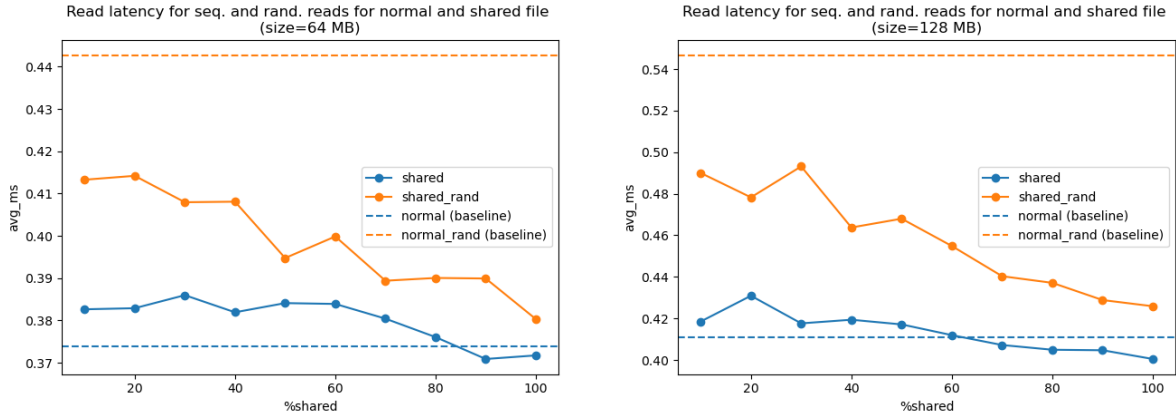


Figure 3: Read latency trends (64 MB and 128 MB)

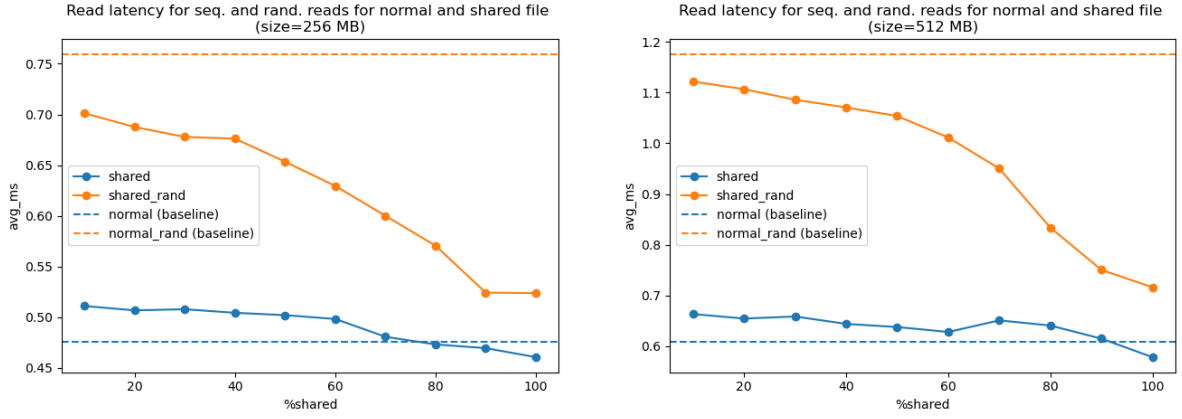


Figure 4: Read latency trends (256 MB and 512 MB)

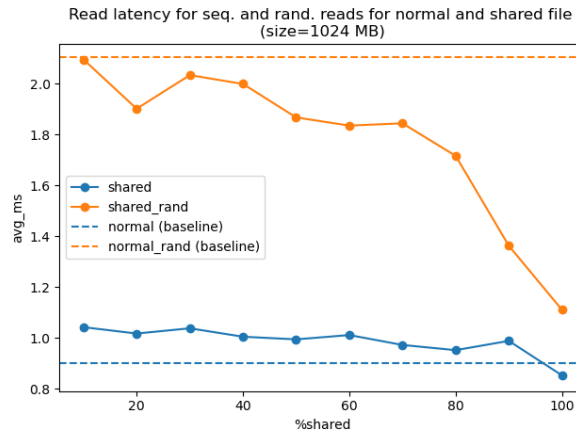


Figure 5: Read latency trend (1024 MB)

8.4.3 Write Benchmarks plots

The write benchmarks evaluate the effect of page deduplication on write latency across different File sizes. Writes to shared pages trigger copy-on-write, where a private copy of the page must be created. This introduces additional overhead, which becomes more noticeable as the proportion of shared pages increases.

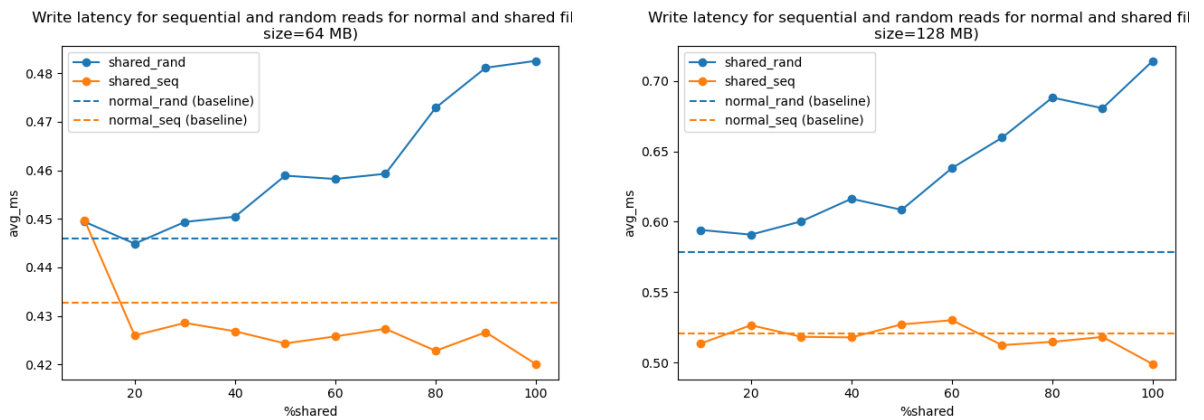


Figure 6: Write latency trends (64 MB and 128 MB)

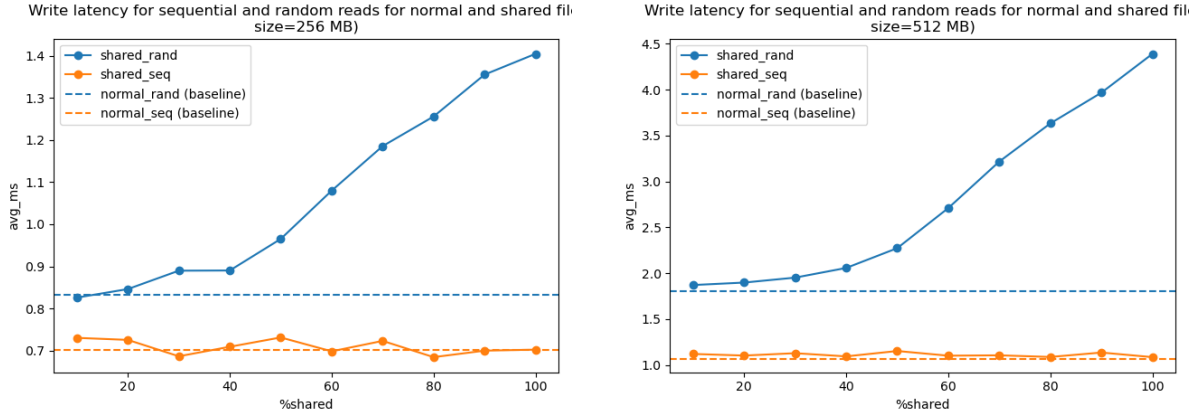


Figure 7: Write latency trends (256 MB and 512 MB)

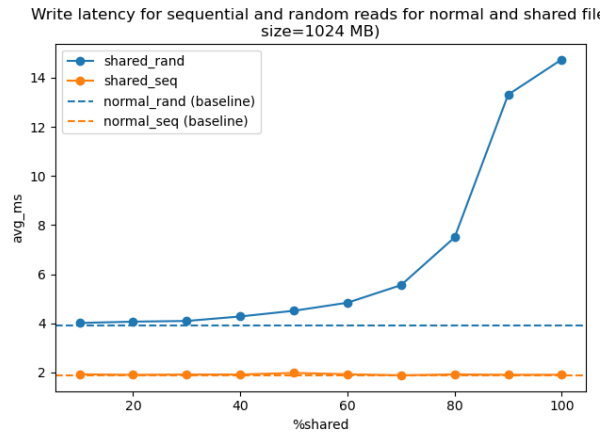


Figure 8: Write latency trend (1024 MB)

8.4.4 Truncate Benchmarks

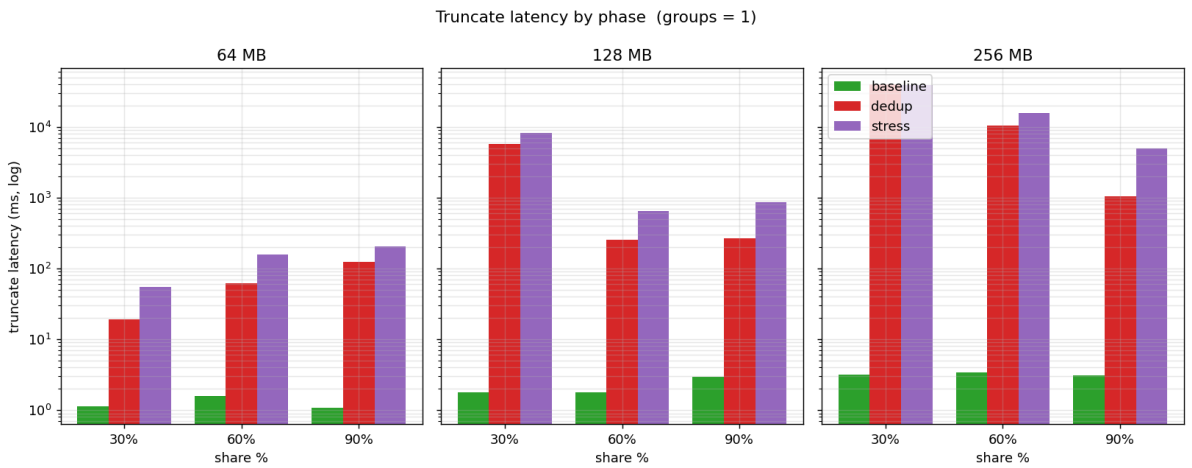


Figure 9: Truncate latency comparison (with percentage shared)

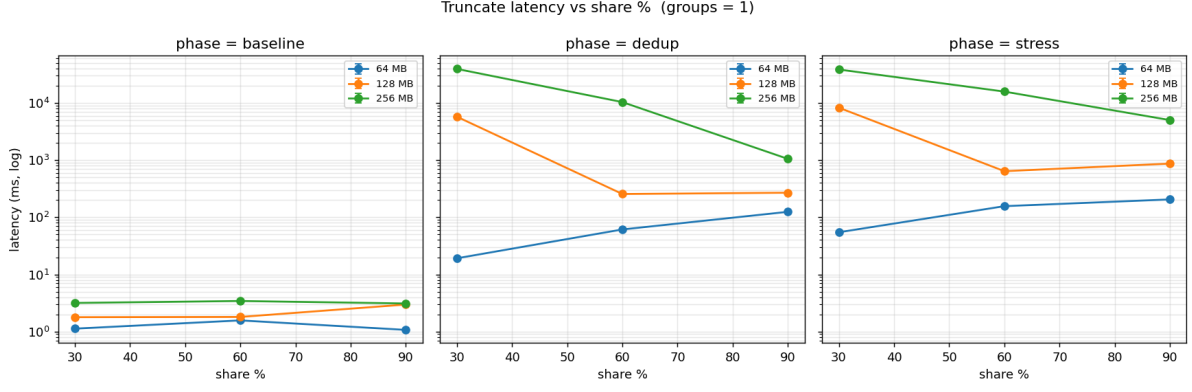


Figure 10: Truncate latency trends (with percentage shared)

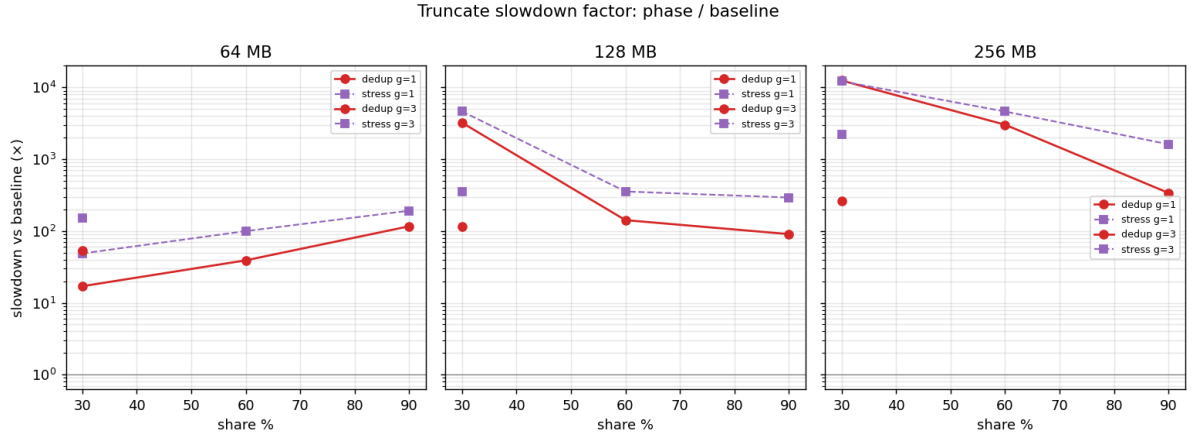


Figure 11: Truncate slowdown trends (with percentage shared)

The phase **baseline** is the phase where our deduplication module is inactive, **dedup** is when pages are deduplicated and are truncated, and **stress** is when **fiio** is writing randomly to some pages and truncate is called at the same time

9 Assumptions and Failures

9.1 Order of folio: 0

Our implementation assumes that the folios are of order 0 in the kernel. The code works only for a 1-1 mapping between folios and pages.

9.2 Page Eviction / Reclaim Integration

As discussed in our initial project meeting, eviction was always contingent on time. Although we set merged pages as unevictable, when the kswapd searches for the mapping pointer, our pages are returned as evictable due to the following calls in the kernel:

```
static inline bool folio_evictable(struct folio *folio)
{
    bool ret;
```

```

/* Prevent address_space of inode and swap cache from being freed */
rcu_read_lock();
ret = !mapping_unevictable(folio_mapping(folio)) &&
      !folio_test_mlocked(folio);
rcu_read_unlock();
return ret;
}

struct address_space *folio_mapping(struct folio *folio)
{
    struct address_space *mapping;

    /* This happens if someone calls flush_dcache_page on slab page */
    if (unlikely(folio_test_slab(folio)))
        return NULL;

    if (unlikely(folio_test_swapcache(folio)))
        return swap_address_space(folio_swap_entry(folio));

    mapping = folio->mapping;
    if ((unsigned long)mapping & PAGE_MAPPING_FLAGS)
        return NULL;

    return mapping;
}

```

folio_mapping returns NULL for our pages, so they are returned as evictable. Below is the image of the call trace of the kernel fault:

```

[+0.399129] BUG: unable to handle page fault for address: 0000001000000040
[+0.000192] #PF: supervisor read access in kernel mode
[+0.000104] #PF: error_code(0x0000) - not-present page
[+0.000098] PSD: 0000000000000000 RS: 0000000000000000 PUD 0
[+0.000100] Dops: 0000 [03] PREEMPT SMP PTI
[+0.000110] CPU: 1 PID: 48337 Comm: python3 Tainted: G      D W O      6.1.4 #23
[+0.000144] Hardware name: QEMU Standard PC (i440FX + PIIX, 1996), BIOS 1.15.0-1 04/01/2014
[+0.000141] RIP: 0010:filemap_release_folio+0x3b/0x70
[+0.000141] Code: 5f 18 e8 88 ff ff 48 8b 00 a8 01 74 ab 48 89 ef a8 79 ff ff 48 8b 00
f6 c4 80 75 30 48 85 db 74 if 48 8b 83 b0 01 00 00 <8> 8b 40 48 48 85 c0 74 0f 5b 44 89 e6 48
89 ef 5d 41 5c ff e0 cc
[+0.000287] RSP: 0018:ffffaa70011df660 EFLAGS: 00010286
[+0.000142] RAX: 00000000ffffffff RBX: ffff958186b6e384 RCK: fffffa70011df708
[+0.000142] RDX: 0000000000000000 RSI: 00000000000141ca RDI: fffff92c4df5680
[+0.000147] RBP: fffff92c4df5680 R08: 0000000000000000 R09: 0000000000000000
[+0.000138] R10: fffff92c4e5e348 R11: 0000000000000000 R12: 00000000000141ca
[+0.000133] R13: 0000000000000000 R14: fffffa70011df79c R15: fffffa70011df6f8
[+0.000133] FS: 00007f1d41671740(0000) GS: fffff9581bcb3000(0000) knlGS:0000000000000000
[+0.000134] CS: 0010 DS: 0000 ES: 0000 CR0: 0000000000050033
[+0.000130] CR2: 0000000100000047 CR3: 00000000026f2005 CR4: 00000000003706e0
[+0.000130] Call Trace:
[+0.000131] <TASK>
[+0.000125] shrink_folio_list+0x8ec/0xd50
[+0.000127] shrink_inactive_list+0x1ab/0x450
[+0.000127] shrink_lruvec+0x435/0x530
[+0.000128] shrink_node+0x1ce/0x440
[+0.000124] do_try_to_free_pages+0xf7/0x420
[+0.000125] try_to_free_pages+0x105/0x320
[+0.000123] __alloc_pages_slowpath.constprop.0+0x366/0xee0
[+0.000129] ? lock_acquire+0x1a4/0x300
[+0.000123] ? rcu_read_lock_sched_held+0x23/0x80
[+0.000124] ? lock_release+0x223/0x2f0
[+0.000122] __alloc_pages+0x343/0x3c0
[+0.000120] filemap_alloc_folio+0x135/0x1c0
[+0.000119] ? lock_release+0x223/0x2f0
[+0.000118] __filemap_get_folio+0x1b4/0x680
[+0.000118] pagecache_get_page+0x15/0xb0
[+0.000116] ext4_da_write_begin+0xc7/0x350
[+0.000126] generic_perform_write+0xc0/0x200
[+0.000115] ext4_buffered_write_iter+0x83/0x130
[+0.000116] ext4_file_write_iter+0x62/0x8a0
[+0.000117] ? lock_acquire+0x1a4/0x300
[+0.000115] ? selinux_file_permission+0x102/0x140
[+0.000118] vfs_writer+0x31f/0x4c0
[+0.000120] ksys_write+0x77/0x140
[+0.000118] do_syscall_64+0x52/0xb0
[+0.000137] entry_SYSCALL_64_after_hwframe+0x63/0xcd

```

9.3 Error while unloading the module

We encountered the following error when the module was unloaded twice during our benchmark scripts. We didn't face this when we unloaded our modules during stress testing. Specifically, while unloading after a truncate on a 512MB file, with 0% sharing. We are unsure if this was an error in the script or what part of our code:

```

[+0.016582] general protection fault, probably for non-canonical address 0xdead00000000108: 0000 [#1]
PREEMPT SMP PTI
[+0.000122] CPU: 1 PID: 33875 Comm: rm Tainted: G      0      6.1.4 #24
[+0.000051] Hardware name: QEMU Standard PC (i440FX + PIIX, 1996), BIOS 1.15.0-1 04/01/2014
[+0.000050] RIP: 0010:release_pages+0x21c/0x390
[+0.000053] Code: 83 c1 01 80 fa 01 48 8b 55 00 19 f6 83 e6 02 83 e2 20 0f 85 45 01 00 00 48 8b 7d 08 4
8 8b 55 10 48 b9 00 01 00 00 00 ad de <48> 89 57 00 48 89 3a 48 89 4d 08 48 83 c1 22 48 89 4d 10 4c 8b
85
[+0.000106] RSP: 0018:ffffae8e0187c70 EFLAGS: 00010002
[+0.000080] RAX: ffff8d4cbfff9000 RBX: ffff8e8e0187da0 RCX: dead000000000100
[+0.000052] RDX: dead000000000122 RSI: 0000000000000003 RDI: dead000000000100
[+0.000050] RBP: ffff8d4cbfff9000 R08: 000000000000001f R09: 0000000000000004
[+0.000052] R10: ffff8d4cbfff9000 R11: ffffffff80000000 R12: 0000000000000002
[+0.000053] R13: ffff8e8e0187e10 R14: 0000000000000001 R15: ffff8d4cbfffc400
[+0.000053] FS:  00007f22613a3580(0000) GS: ffff8d4cbbc80000(0000) knlGS: 0000000000000000
[+0.000064] CS:  0010  DS:  0000  ES:  0000  CR0: 0000000080050033
[+0.000051] CR2: 000059e2c7c31728 CR3: 0000000102dfc006 CR4: 00000000003706e0
[+0.000052] Call Trace:
[+0.000053] <TASK>
[+0.000049] __pagevec_release+0x16/0x30
[+0.000050] truncate_inode_pages_range+0x163/0x4e0
[+0.000052] ext4_evict_inode+0x170/0x6a0
[+0.000054] evict+0x9c/0x160
[+0.000053] do_unlinkat+0x1c6/0x2c0
[+0.000052] __x64_sys_unlinkat+0x2f/0x60
[+0.000050] do_syscall_64+0x52/0xb0
[+0.000054] entry_SYSCALL_64_after_hwframe+0x63/0xcd
[+0.000062] RIP: 0033:0x7f22612bfff2b

```

10 Future Work and Optimisations

- **KSM-style 256-page cap per node.** The scanner currently permits arbitrarily large reverse-map lists. Capping each node and chaining overflow nodes would bound the per-page split and unmerge cost.
- **Tree-based stable list.** The current $O(N)$ linear scan of the global stable list dominates scanner CPU when the working set is large. Replacing it with a red-black tree keyed on the page hash would reduce this to $O(\log N)$.
- **mmap / eviction / folios of larger orders**

11 Conclusion

We have delivered a working in-tree page-cache deduplication subsystem that:

- detects duplicate content with a JHash2-based KSM-style two-list scanner;
- merges duplicates by retargeting xarray entries to a shared survivor page;
- splits survivors back to private copies on writes via a single direct hook in `__filemap_get_folio`;
- correctly handles `ftruncate` and `drop_caches` through dedicated hooks in `mm/truncate.c` and `fs/drop_caches.c`;
- passes a 15-test suite covering basic correctness, multi-threaded read/write, mixed concurrent operations (read + write + truncate + cp), refcount stress, and large-scale dedup — all with byte-exact post-test verification both in cache and on disk.

Empirical memory savings closely approach the theoretical maximum on duplicate-rich workloads (e.g. 1000 identical files reduced from ~16 MB to <100 KB of cache), demonstrating that page-cache deduplication is a viable memory-efficiency technique for systems with high content redundancy in file-backed pages.